

ANALYSIS OF HOW APPLICATIONS SANDBOX WORKS ON SECURITY SYSTEM OF MacOS



Arranged by:

Arla Bukit Elbirun	2521010177
Arli Fabian	2505424011
Farisa Aulia Sudrajat	2521010184

CEP-CCIT
FACULTY OF ENGINEERING
UNIVERSITY OF INDONESIA

2025

Preface

In the name of Allah SWT. Praise and gratitude for His presence, mercy, and guidance so that the author can complete this writing task well and smoothly. This paper has been compiled by the author to the best of his ability and with the assistance of several parties. Therefore, the author would like to express his gratitude to those who contributed to the completion of this writing assignment. The author realizes that there are still many shortcomings and errors in this writing assignment. Therefore, we are open to any criticism and suggestions from readers so that the author can improve the writing.

TABLE OF CONTENTS

CHAPTER I INTRODUCTION	5
1.1 Background.....	5
1.2 Writing Objectives	5
1.3 Problem Domain	5
1.4 Writing Methodology	6
1.5 Writing Framework	6
CHAPTER II BASIC THEORY	7
2.1 What is Operating System Security?.....	7
2.2 Security Techniques in MacOS	7
2.3 Three Primary Security Threats in Operating System	8
2.4 Relationships Between Three Primary Security Features and MacOS Sandbox .	9
CHAPTER III PROBLEM ANALYSIS	12
3.1 How Sandbox Works on macOS.....	12
3.2 Sandbox Working Mechanism: Kernel Architecture	14
3.3 The Strategic Role of Sandboxing: Damage Control and Zero Day Defense....	15
3.4 The Effectiveness of Sandbox in Addressing Security Threats	16
3.5 Sandbox Implementation for Permission Management	17
CHAPTER IV CONCLUSION AND SUGGESTION	20
4.1 Conclusion.....	20
4.2 Suggestion	20
BIBLIOGRAPHY	21

TABLE OF FIGURES

Figure 2.2 App SandBox.....	7
Figure 2.3 Malware	8
Figure 2.3 Data Privacy Breaches	9
Figure 2.3 System Integrity Violations.....	9
Figure 2.4 SandBox Malware Attacks	10
Figure 2.4 SandBox On Privacy Protection.....	10
Figure 3.1 MacOS SandBox Architecture	12
Figure 3.2 SandBox Working Mechanism: Kernel Architecture.....	14
Figure 3.2 MacOS Camera Access Permission.....	14

CHAPTER I INTRODUCTION

1.1 Background

In today's world, operating system technology continues to evolve. Support applications that work on operating systems continue to evolve. It is important for applications to maintain their security. Therefore, support applications are needed to secure applications.

On the iOS operating system, support applications are needed for security. The supporting application for this operating system is the SandBox application on MacOS. SandBox is responsible for restricting application access to system resources and user data by limiting it to a restricted environment.

The system works through controlled access, a unique environment, privilege restrictions, and explicit permissions. The SandBox application implementation will work automatically on the MacOS & iOS operating systems. This ensures that applications on these operating systems are well protected. Therefore, this paper was written to discuss how the SandBox application works on MacOS to support the MacOS security system.

1.2 Writing Objectives

The purpose of this paper is to explain how SandBox Applications works on MacOS and how they operate. This includes a How does the sandbox application work in setting mandatory access control policies on macOS 10.15 and explain which limits access to the data that can be accessed by the application. The main focus will be on how SandBox applications work on the MacOS operating system and explain how sandbox overcomes problems like malware threat, data privacy, and system integrity.

1.3 Problem Domain

This study examines how SandBox applications and MacOS work together to secure user data against three primary security threats: malware attacks, data privacy breaches, and system integrity violations. SandBox applications are automatically available on every application on MacOS, creating mandatory isolation that prevents malware from

spreading, restricts unauthorized data access, and protects critical system files from modification. This is different from the Android operating system, which does not have SandBox applications and cannot automatically provide them with the same level of enforcement.

The purpose of this study is to identify how SandBox application security works on MacOS in addressing these security challenges—specifically in securing and restricting access to sensitive resources such as camera, microphone, Bluetooth, and contacts. This research analyzes how the Sandbox mechanism effectively contains malware within application containers, enforces data privacy through permission controls, and maintains system integrity via kernel-level protection.

1.4 Writing Methodology

This study uses a descriptive-analytical approach with a literature review method to analyze the working mechanism of the Sandbox application on the macOS security system.

1.5 Writing Framework

Chapter I: Introduction

MacOS and the SandBox security application used by MacOS to provide readers with an overview.

Chapter II: Basic Theory

Introduce readers to the basic material about MacOS and SandBox Applications, which will be explained further in the following chapters. This material will cover the security components of the operating system in MacOS, such as how SandBox Applications work and their role in MacOS security.

Chapter III: Problem Analysis

Problems in the MacOS and SandBox Applications to improve security systems.

Chapter IV: Conclusion and Suggestion

Conclusions about how security systems work on MacOS and suggestions for the future.

CHAPTER II BASIC THEORY

2.1 What is Operating System Security?

Operating system security is a series of steps and actions aimed at isolating computer operating systems from various threats, such as viruses, malware, hackers, and unauthorized access. Operating system security is a fundamental aspect of modern computing that aims to protect systems, data, and users from internal and external threats.

The main principle in operating system security is the principle of least privilege, which states that every program or user must operate with the minimum access rights necessary to complete their tasks. In the context of applications, this principle requires an isolation mechanism that can limit an application's access to system resources. This isolation mechanism is known as sandboxing.

2.2 Security Techniques in MacOS

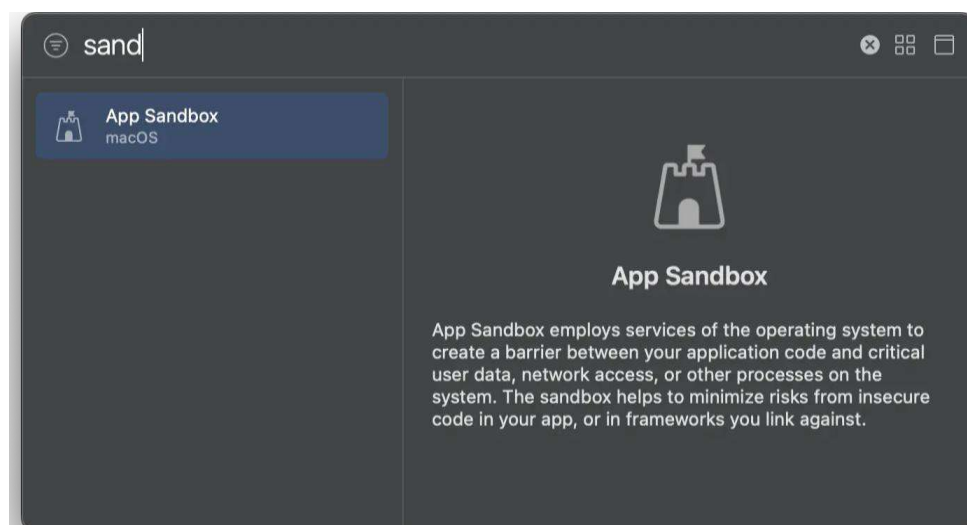


Figure 2.2 App Sandbox

(Ref: <https://docs.amplify.aws/gen1/swift/build-a-backend/push-notifications/set-up-push-notifications/>)

The security technique used by macOS is application sandboxing. Application sandboxing is a security technology that restricts applications' access to system resources, user files, and other applications. Sandboxing creates an isolated

environment in which applications run with the minimum access rights necessary to perform their functions.

MacOS app sandbox is a security framework implemented by Apple to protect the system and user data from potentially dangerous applications. This framework was first introduced in MacOS X Lion (10.7) in 2011 and has been a mandatory requirement for all applications distributed through the Mac App Store since March 2012.

Sandboxing is a security technique that creates an isolated and controlled environment for running applications. The term “sandbox” is analogous to a children's sandbox, where activities are limited to a specific area and do not affect the surrounding environment.

2.3 Three Primary Security Threats in Operating System

In operating systems, there are three main security threats, namely:

1. Malware Attacks



Figure of 2.3 Malware

(Ref: <https://www.alliantcybersecurity.com/different-types-malware-attacks-cybersecurity/>)

Malware (malicious software) is harmful software designed to damage, exploit, or gain unauthorized access to computer systems and user data.

2. Data Privacy Breaches



Figure of 2.3 Data Privacy Breaches

(Ref: <https://studv.com/academy/lesson/data-breaches-definition-statistics.html>)

A data privacy breach is unauthorized access, collection, use, or dissemination of users personal or sensitive information without explicit consent.

3. System Integrity Violations

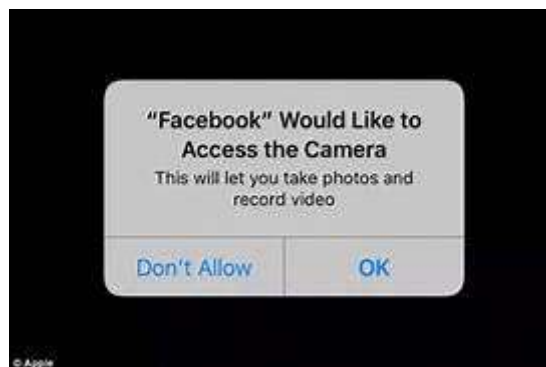


Figure of 2.3 System Integrity Violations

(Ref: <https://www.axxonsoft.com/products/video-management-software/key-features/cybersecurity-and-data-protection>)

A system integrity violation is an unauthorized modification to system files, configurations, or critical components that could compromise the stability, security, or functionality of the operating system.

2.4 Relationships Between Three Primary Security Features and MacOS Sandbox

This subchapter explains how sandboxing addresses these three security threats on MacOS.

1. SandBox Against Malware Attacks



Figure of 2.4 SandBox on Malware Attacks

(Ref: [What is Malware? How to Prevent Malware Attacks? | Fortinet](#))

An example of an attack on malware is Container Isolation. The process includes:

- A. When the application is installed → macOS creates a unique container
- B. Each application gets its own isolated directory
- C. The application can only access files within its container.
- D. Unable to access other application containers

Therefore, from this process, it can be concluded that application 1 cannot access the folder of application 2.

2. SandBox on Privacy Protection

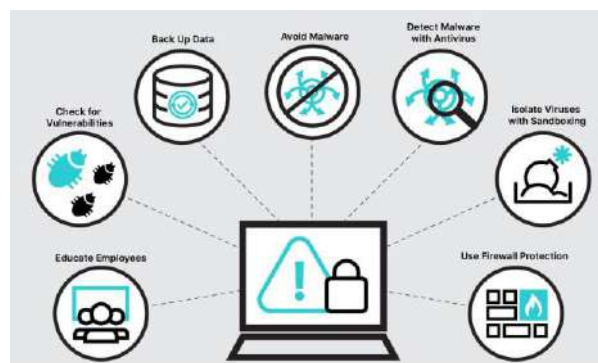


Figure of 2.4 SandBox on Privacy Protection

(Ref: <https://www.dailymail.co.uk/sciencetech/article-5020769/iPhone-apps-silently-turn-camera.html>)

An example of an attack on Data Privacy Breaches is Runtime Authorization. The process includes:

- A. Applications compiled with entitlements. (Example : camera)
- B. User opens the application
- C. First-time camera access request application

- D. macOS pause application → bring up dialog box
- E. User select: “Allow” or “Don't Allow”
- F. User choice is stored in the permission database
- G. Next request: check the database first

Then the dialog box that will appear is “App.app” would like to access the camera
[Don't Allow] or [OK]

CHAPTER III PROBLEM ANALYSIS

As explained in the problem domain, there are three main threats that macOS users often face: malware, data privacy, and system integrity damage. Malware can infiltrate through infected applications and potentially damage or steal system data. Data privacy occurs when applications access sensitive information without clear permission, such as access to the camera, microphone, or contacts. Meanwhile, system integrity damage can occur if malicious applications successfully modify important files from the operating system.

To address these three threats, macOS has a security mechanism namely Sandbox that works through several layers of protection. Sandbox isolates each application in a separate container, limits its access rights through entitlements, and requires applications to request permission directly from users before accessing sensitive resources. This chapter will discuss how Sandbox works technically, its security architecture at the kernel level, its strategic role in protecting the system, and its real-world implementation in managing application permissions. By understanding this mechanism, we can see how Sandbox effectively handles security challenges in macOS.

3.1 How Sandbox Works on macOS

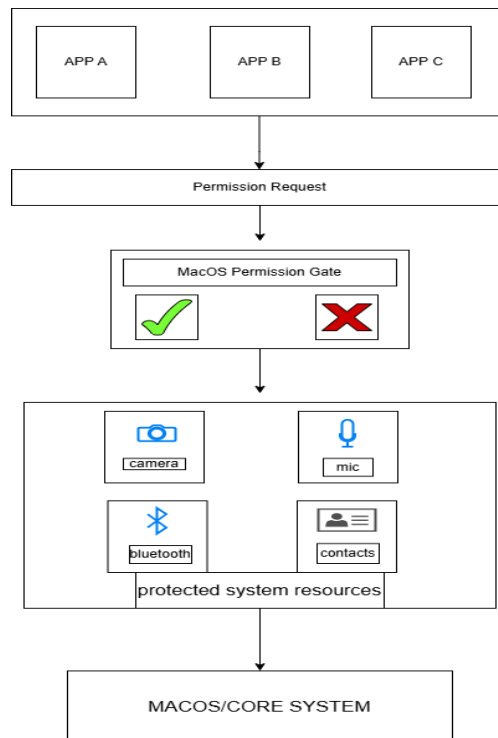


Figure 3.1 MacOS Sandbox Architecture

Sandbox is one of the core security mechanisms of macOS in constraining an application's access to system resources and user data. The basic principle of sandbox is containerization where each application runs in an environment separate from other applications and the operating system.

The sandbox mechanism starts right from the installation of an application. Each application is assigned a unique identifier by the macOS system and executed in a dedicated process. Applications can only access resources that have been explicitly granted permission through the entitlements mechanism. Entitlements are in the form of digitally signed key-value pairs, so they cannot be tampered with after the compilation of the application. This prevents the manipulation of permissions granted during runtime.

To harden the sandbox, macOS has numerous other layers of protection. First, the system partition is mounted read-only to prevent applications from being able to make changes to system files. All third-party applications run as unprivileged users, instead of running as root, which can potentially reduce the possibility of privilege escalation in case an application is compromised. With this mechanism, damage by infected applications will remain under control and isolated within the application's sandbox container.

What sets the macOS sandbox apart from all other platforms is the implementation of a consent-based access system. This means that users must explicitly consent whenever different applications require access to hardware such as the camera or microphone, and access to user data like contacts or location. Permissions like these are easily revoked through System Settings at any time, thus giving the users full control over their data and privacy.

3.2 Sandbox Working Mechanism: Kernel Architecture

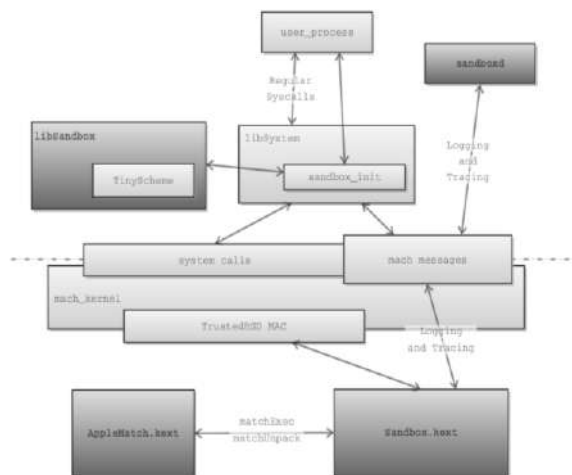


Figure 3.2 Sandbox Working Mechanism: Kernel Architecture
(Ref: [apple-sandbox.pdf](#))

Sandboxing enforcement in macOS works at the deepest level of the operating system, namely the XNU kernel, which can be considered the ‘brain’ of the computer. Because it works at this level, the protection it provides is very fundamental and at a low level of the system.

Technically, this Sandbox is a policy module installed into a framework called TrustedBSD Mandatory Access Control (MAC) Framework. This MAC framework is the main and highest security system in macOS. It is like a layered security system installed directly at the core.

The way Sandbox works is quite simple. Any application running under the macOS needs to request permission from the kernel if it wants to do something, such as reading files or accessing the internet. It does so through a system called a system call. There is a module, known as Sandbox.kext, which is responsible for intercepting or monitoring all incoming system calls. So every application request is monitored first.

After the request is intercepted, MACF enforces verification as to whether an application has the right to perform the action in question. For example, when application A requests an access operation on data belonging to application B, MACF will check its permissions based on the applied Sandbox rules. If the action goes against the rules, the kernel blocks the operation, sending an EPERM error code to the

application. In such a system, security controls are mandatory and cannot be ignored or manipulated by any application.

By working at the kernel level and using a strict MACF architecture, the macOS sandbox has a strong technical foundation for protecting the system. This low-level mechanism not only functions as a normal access restriction, but also plays a strategic role in defending the system against more complex security threats.

3.3 The Strategic Role of Sandboxing: Damage Control and Zero Day Defense

A zero-day vulnerability is a security flaw that is unknown to software developers, leaving no time to fix it before it is exploited. This vulnerability is considered the most dangerous threat to cybersecurity for three main reasons: first, there are no updates available to close the security gap; second, antivirus software cannot detect the attack because it does not recognize it; third, hackers already know about the vulnerability, allowing them to launch attacks without effective defense mechanisms. For example, if there is a hidden bug in Safari that Apple is unaware of, hackers can attack millions of devices without any prevention mechanism. This causes zero-day vulnerabilities to be frequently traded on the dark web for millions of dollars and become the main weapon in high-level cyber attacks.

The role of Sandboxing on macOS is not only to confine the access of applications, but also to serve as the main protection against attacks: even in cases of zero day attacks that is, attacks taking advantage of unknown security vulnerabilities, though the attacker successful in entering the application, will remain confined within the application container and cannot spread to other systems or applications.

This strategy uses the “Default Deny” principle, which means that all application actions are automatically prohibited first. Applications can only perform actions that have been explicitly permitted through entitlements. So even if there are new security vulnerabilities, applications still cannot do anything outside of the permissions that have been set.

What makes the macOS sandbox different from other platforms is its user-based permission system. Every time an application wants to access sensitive information such as the camera, microphone, location, or personal data, it must request permission

directly from the user. Users have full control and can revoke permission at any time through System Settings. This reduces the risk of data theft or privacy violations.

3.4 The Effectiveness of Sandbox in Addressing Security Threats

The effectiveness of the macOS sandbox can be seen from several important points that have been explained in the previous chapter:

1. Application Isolation Capabilities

Effective with Reservations based on the containerization architecture described above, each application runs in a separate container with limited access. This mechanism has proven effective in limiting the spread of malware because applications cannot access other application containers. However, recent research has found various overlooked attack surfaces, particularly in XPC services that lack entitlement checks or sandbox checks for incoming clients, resulting in the discovery of various CVEs such as CVE-2023-27944, CVE-2023-32414, and CVE-2023-42961. This shows that while application isolation is generally effective, there are still security gaps that can be exploited by sophisticated attackers. A concrete example of this weakness is CVE-2022-26706, which allows malicious code to escape from the App Sandbox and run without restrictions on the ResearchGate system. Although this vulnerability has been patched by Apple, this discovery proves that sandboxes are not perfect systems and require continuous security updates to close any gaps that are found.

2. Enforcement di Kernel Level

Strong with Gap in Adoption The enforcement sandbox that works at the kernel level through the Trusted BSD MAC Framework provides strong protection because applications cannot bypass the established policy. Every system call is verified by Sandbox.kext before execution, and unauthorized access is immediately blocked with an error code. This mechanism is technically very robust in preventing privilege escalation. However, the effectiveness of this enforcement depends on developers' adoption of the sandbox. Although 94% of Mac App Store applications are required to use the sandbox, more than 89% of applications from third-party stores do not use it. This significant adoption gap creates a situation where users who install applications from outside the Mac App Store do not actually get any sandbox protection at all. This

shows that the effectiveness of sandboxing in macOS is contextual: it is very effective for App Store applications, but ineffective for third-party applications that are not sandboxed.

3. User Control over Sensitive Data

Effective with Usability Trade-offs The consent-based permission system gives users full control over the app's access to sensitive data such as the camera, microphone, Bluetooth, and contacts. Users can grant or revoke permissions at any time through System Settings, and the sandbox immediately blocks access when permissions are revoked. This mechanism has proven effective in protecting user privacy because the final decision rests with the user, not the app. However, the effectiveness of this system has limitations that are human factors. Too many permission prompts can cause “permission fatigue” where users tend to approve all permissions without reading carefully. In addition, an arbitrary folder creation vulnerability was found that allows bypassing sandbox restrictions for folder creation, indicating that the implementation of the permission system still has edge cases that have not been handled perfectly.

3.5 Sandbox Implementation for Permission Management

The macOS-based sandbox uses a permissions system to control application access to sensitive data and devices. These permissions are known as entitlements, or the access rights to be registered by developers when developing applications. The permissions will be checked and validated by Sandbox when the application is running. In this paper, there are four main permissions. That will be analyzed including the camera, microphone, Bluetooth, and contacts.

These four resources were chosen because they are very risky if misused. Unauthorized access to the camera and microphone can be used to spy on users secretly. Bluetooth can be exploited in tracking or data theft. Contacts contain highly sensitive personal information such as names, telephone numbers, and e-mail addresses.



Figure 3.2 MacOS camera access permission

(Ref: <https://share.google/images/9Orl4PO6fnw6tbcQH>)

1. Camera

Camera Permission uses the **com.apple.security.device.camera** entitlement. That provides access not only to built-in cameras but to external ones too. If an app doesn't have this entitlement, it cannot access the camera at all. If such an app attempts to use the camera for the first time, the system opens a dialog box where the user can give consent. The sandbox checks every attempt to access the camera. If the application does not have the correct permissions, the sandbox immediately blocks the access before it reaches the camera device.

2. Microphone

Microphone Permission uses the same **com.apple.security.device.microphone** entitlement as camera permission. An application must have this entitlement and request user permission before it can record audio. For transparency, macOS shows an indicator in the menu bar when the microphone is in use.

3. Bluetooth

Bluetooth Permission uses the code **com.apple.security.device.bluetooth** to access Bluetooth devices. Sandbox blocks any connection or attempt to communicate with Bluetooth devices if the application does not have this permission. This is important because it does not allow applications to connect or communicate with Bluetooth devices without users' knowledge. Which later on could be used for data or location tracking.

4. Contacts

Contacts Permission uses the code `com.apple.security.personal-information.addressbook` for access to the contact list. Without this permission, the application cannot read or modify contacts. The sandbox prevents access to the contact database if In cases where the application does not have this permission, personal information includes names. And phone numbers remain protected. It enables a user to set such permission in System Settings > Privacy & Security and cancel access at any time without needing to uninstall the application.

CHAPTER IV CONCLUSION AND SUGGESTION

4.1 Conclusion

Based on the analysis conducted, the macOS sandbox demonstrates significant but imperfect effectiveness in addressing malware threats, protecting data privacy, and maintaining system integrity.

Containerization and application isolation have proven effective in limiting the spread of malware, although recent research has found several vulnerabilities such as CVE-2022-26706 and CVE-2023-27944 that show that the sandbox can be bypassed under certain conditions. The kernel-level sandbox architecture with its “Default Deny” principle provides strong defense against zero-day attacks, but its effectiveness depends on developer adoption—while 94% of Mac App Store applications use sandboxing, more than 89% of third-party applications do not. The user consent-based access system provides full control over personal data through transparent permission management, despite limitations related to human factors such as permission fatigue.

Overall, the macOS sandbox is one of the most robust security mechanism implementations on modern desktop platforms. Although it is not immune to advanced attacks and relies on developer adoption and user awareness, the sandbox has proven effective in preventing widespread malware infections and protecting user data through a comprehensive multi-layer defense mechanism.

4.2 Suggestion

It is recommended to study Apple official documentation on App Sandbox and the entitlements system. In addition, further exploration of the TrustedBSD MAC Framework and XNU kernel architecture can provide deeper technical insights. With a deeper understanding of sandbox mechanisms, we can be more concerned about operating system security aspects and their implementation in developing more secure applications.

BIBLIOGRAPHY

Apple Inc. *Accessing Files from the macOS App Sandbox*. Apple Developer Documentation. Retrieved from <https://developer.apple.com/documentation/security/accessing-files-from-the-macos-app-sandbox>

Apple Inc. *App Sandbox*. Apple Developer Documentation. Retrieved from <https://developer.apple.com/documentation/security/app-sandbox>

Apple Inc. *com.apple.security.files.user-selected.read-only Entitlements*. Apple Developer Documentation. Retrieved from <https://developer.apple.com/documentation/bundleresourcesentitlementscom.apple.security.files.user-selected.read-only>

Apple Inc. *Configuring the macOS App Sandbox*. Xcode Documentation. Retrieved from <https://developer.apple.com/documentation/xcode/configuring-the-macos-app-sandbox>

Apple Inc. *Keamanan Platform Apple*. Apple Support. Retrieved from <https://support.apple.com/id-id/guide/security/secabd3504cd/web>

Apple Inc. *Security of Runtime Process*. Apple Platform Security Guide. Retrieved from <https://support.apple.com/guide/security/security-of-runtime-process-sec15bfe098e/web>

Blazakis, D. (2011). *The Apple Sandbox*. Retrieved from <https://www.ise.io/wp-content/uploads/2017/07/apple-sandbox.pdf>

Census Labs. (2019). *Sandbox: ARGP CSW 2019*. Retrieved from <https://census-labs.com/media/sandbox-argp-csw2019-public.pdf>

IBM. *Application Security*. IBM Think Topics. Retrieved from <https://www.ibm.com/id-id/think/topics/application-security>

Payrix.*Sandbox Apple Pay Testing. Payrix Documentation.* Retrieved from <https://resource.payrix.com/docs/sandbox-apple-pay-testing>

Point.co.*Understanding App Sandboxing on iOS: How It Safeguards User Data.* Retrieved from <https://point.co/understanding-app-sandboxing-on-ios-how-it-safeguards-user-data>

TrustedBSD Project.*Mandatory Access Control (MAC).* Retrieved from <http://www.trustedbsd.org/mac.html>



ANALYSIS OF HOW APPLICATIONS SANDBOX WORKS ON SECURITY SYSTEM OF MACOS

Group 2:

- Arla Bukit Elbirun (2521010177)
- Arli Fabian (2505424011)
- Farisa Aulia Sudrajat (2521010184)

Start It Now





INTRODUCTION

Background

- Operating systems continue to evolve
- Application security is crucial
- macOS uses Sandbox to restrict application access to system and user data

Objectives Explain how Sandbox works on macOS in addressing:

- Malware attacks
- Data privacy breaches
- System integrity violations





New Challenges in the Digital World

OPERATING SYSTEM SECURITY



Main Principle: Least Privilege

- Each program/user only has minimum necessary access
- Requires application isolation mechanism

Sandbox on macOS

- First introduced: macOS X Lion (10.7) – 2011
- Mandatory for Mac App Store since 2012
- Creates isolated environment for each application



Reasons Why Security Is Critical



1. Malware Attacks → Container Isolation

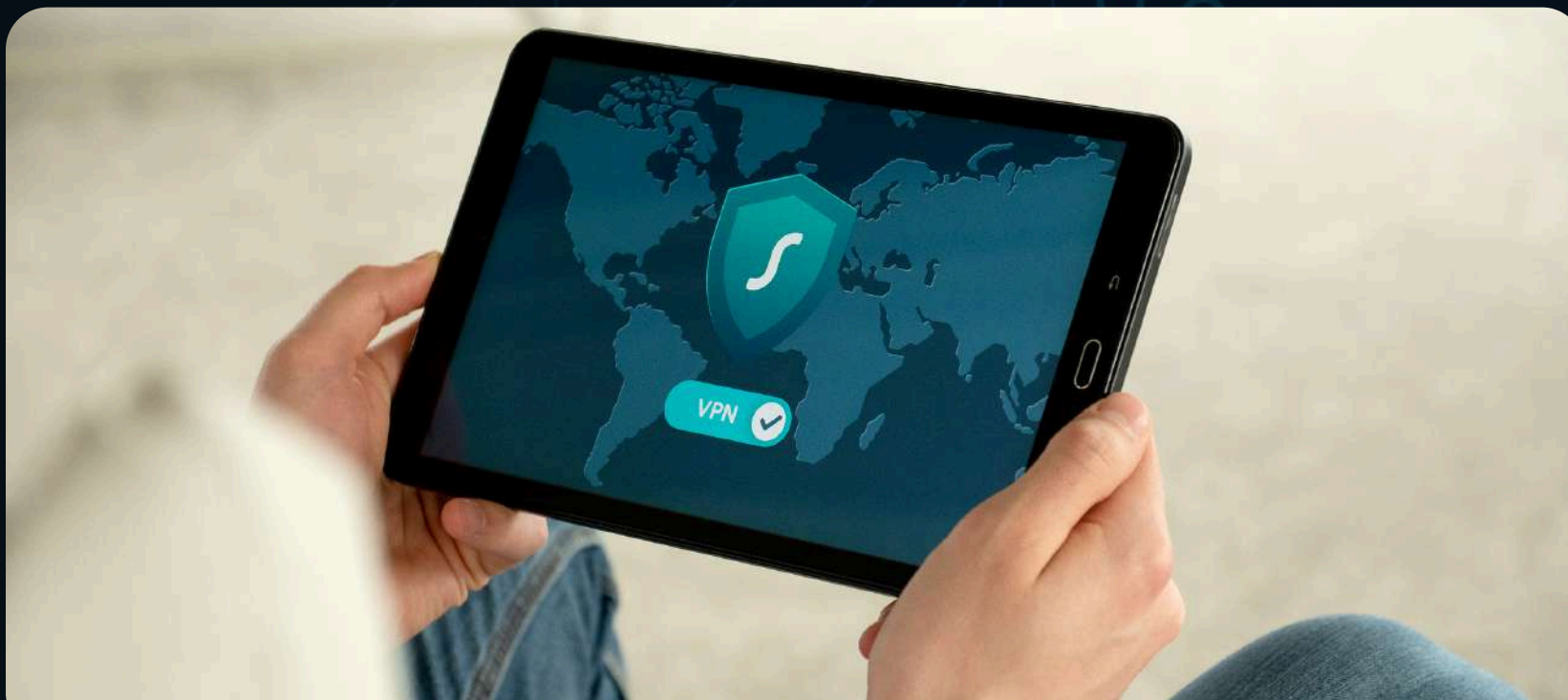
- Each app in separate unique container
- App only accesses files within its own container
- Result: Malware cannot spread to other apps

2. Data Privacy Breaches → Runtime Authorization

- App requests permission on first-time access
- macOS pauses app → dialog box appears
- User chooses "Allow" or "Don't Allow"
- Result: Full user control over sensitive data

3. System Integrity Violations → Read-Only System

- System partition mounted read-only
- Apps run as unprivileged users
- Result: Apps cannot modify system files



SECURITY THREATS & SANDBOX SOLUTIONS

**1. Containerization**

- Each application runs in separate environment
- Unique identifier for each app

2. Entitlements

- Digitally signed key-value pairs
- Cannot be modified after compilation

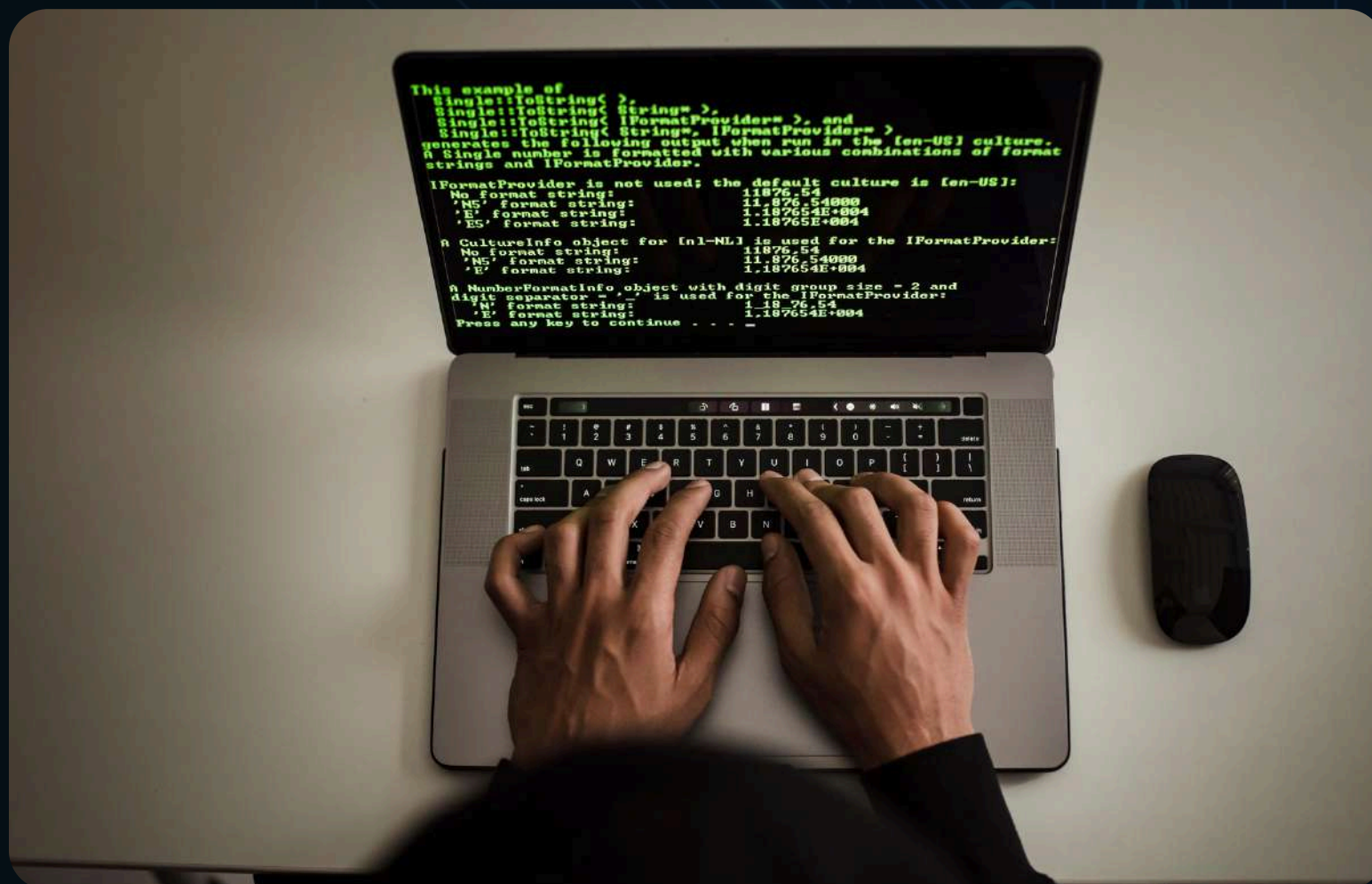
3. Consent-Based Access

- User must explicitly consent for hardware/data access
- Permissions can be revoked anytime

HOW SANDBOX WORKS



Sandbox operates at XNU
Kernel level



KERNEL ARCHITECTURE

TrustedBSD MAC Framework:

- Sandbox.kext intercepts all system calls
- MACF verifies permissions before execution
- If rules violated → kernel blocks with EPERM error

Benefits:

- Mandatory security control
- Cannot be manipulated by applications
- Fundamental protection at low-level system

STRATEGIC ROLE



Building a Defense System

Default Deny Principle:

- All application actions automatically prohibited
- Only actions with explicit permission allowed

Zero Day Defense:

- Even if attacker enters, remains confined in container
- Cannot spread to system/other applications

User Control:

- Full control over personal data
- Revoke permissions anytime via System Settings

PERMISSION MANAGEMENT

Main Permissions:

1. Camera - `com.apple.security.device.camera`
2. Microphone - `com.apple.security.device.microphone`
3. Bluetooth - `com.apple.security.device.bluetooth`
4. Contacts - `com.apple.security.personal-information.addressbook`

All require:

- Entitlement declaration
- User consent on first-time access
- Manageable via System Settings



PERMISSION MANAGEMENT





CONCLUSION

3 Key Success Aspects of Sandbox:

1. Containerization & Isolation

Damage localized within container

2. Kernel-Level Architecture

- Default Deny principle
- Protection against zero-day attacks

3. User-Based Access System

- Transparent permission management
- Full user control over privacy





THANK YOU



Thank you for your attention
